

Contrôle de robot ESP32 via interface web

✱ claude.ai/chat/2455320f-87f6-48b4-b2f6-cdc019426fb5



a partir d'une interface web j'agis sur les paramètres PID ainsi que sur TA (target angle =90°) un joystick me permet de faire avancer/ reculer/tourner le robot. Un terminal intercepte les messages de la ligne série et les affiche sur la page web. Le robot est programmé en C++ et le contrôleur est un ESP32

29 mars

Je vais créer une interface refaite et améliorée pour ton contrôleur de robot ESP32, avec un look "panneau de contrôle industriel" bien plus soigné.

Voici l'interface refaite ! Voici ce qu'elle fait :

Panneau PID — chaque paramètre (TA, KP, KI, KD) a ses champs Min/Max éditables, un slider proportionnel, et un afficheur de valeur live. Modifier le min/max recalibre automatiquement le slider.

Joystick — les boutons directionnels (▲ ▼ ◆) incrémentent posX/posY en continu tant qu'ils sont pressés. Tu peux aussi utiliser les **touches fléchées du clavier** + espace pour stopper. Le bouton ■ central remet à zéro.

Terminal série — chaque action (changement PID, joystick, wifi, sauvegarde) génère une ligne horodatée avec coloration sémantique (vert = OK, orange = warning, rouge = erreur).

Pour l'intégration côté ESP32, les commandes WebSocket pourraient suivre ce schéma JSON :

json

```
{ "cmd": "pid", "TA": 90, "KP": 1.2, "KI": 0.05, "KD": 0.3 }
{ "cmd": "move", "dir": "FWD", "x": 0, "y": 50 }
{ "cmd": "save" }
```

Tu veux que j'adapte le format des messages ou ajouter des fonctionnalités (graphe temps-réel de l'angle, indicateur de batterie, reconnexion auto) ?

je prefere un joystick que des boutons directionnels.

29 mars

Voilà ! Le joystick est maintenant analogique et draggable :

- **Glisser** le stick dans n'importe quelle direction — il reste dans la zone circulaire
- **Relâcher** → retour au centre avec une animation de rappel progressif
- Les valeurs **posX / posY** vont de -100 à +100 en continu
- La direction détectée (**FWD / BCK / LEFT / RIGHT / STOP**) s'affiche en temps réel et se logue dans le terminal
- Fonctionne aussi au **tactile** sur mobile/tablette

J'aurais préféré un theme plus clair pour l'interface

29 mars

Voilà, thème clair appliqué ! La palette utilise des bleus doux (`#378ADD`, `#185FA5`) sur fonds blancs et gris très légers, avec le terminal en bleu pâle au lieu du noir — tout en gardant la lisibilité des logs (vert pour OK, orange pour warning, rouge pour erreur). Le joystick garde son effet de profondeur avec un dégradé bleu.

si dans mon programme c++ j'ai des variables TA,KP,KI,KD de type float je voudrais qu'a chaque modification d'un curseur, la valeur modifiée de ce curseur soit affectée à la variable correspondante dans C++

06:13

La communication se fait via **WebSocket**. Voici les deux côtés :

Côté JavaScript — à chaque changement de slider, on envoie un message JSON :

javascript

```
sl.addEventListener('input', () => {
  const v = parseFloat(sl.value).toFixed(dec);
  vd.textContent = v;

  // Envoi WebSocket vers l'ESP32
  if (ws && ws.readyState === WebSocket.OPEN) {
    ws.send(JSON.stringify({ param: p.id, value: parseFloat(v) }));
  }
});
```

Et la connexion WebSocket à initialiser une fois au chargement :

javascript

```
const ws = new WebSocket('ws://192.168.1.42/ws');
ws.onmessage = e => addLog('[WS] ' + e.data, '');
ws.onclose = () => addLog('[WS] Déconnecté', 'warn');
ws.onopen = () => addLog('[WS] Connecté', 'ok');
```

Côté C++ ESP32 — réception et affectation aux variables float :

cpp

```

#include <Arduino.h>
#include <WiFi.h>
#include <ESPAsyncWebServer.h>
#include <ArduinoJson.h>

// Tes variables globales
float TA = 90.0f;
float KP = 1.2f;
float KI = 0.05f;
float KD = 0.30f;

AsyncWebServer server(80);
AsyncWebSocket ws("/ws");

void onWsEvent(AsyncWebSocket *server, AsyncWebSocketClient *client,
               AwsEventType type, void *arg, uint8_t *data, size_t len) {

    if (type == WS_EVT_DATA) {
        AwsFrameInfo *info = (AwsFrameInfo *)arg;
        if (info->opcode == WS_TEXT) {
            data[len] = '\0';

            StaticJsonDocument<128> doc;
            if (deserializeJson(doc, (char *)data) == DeserializationError::Ok) {

                const char* param = doc["param"];
                float value = doc["value"];

                if (strcmp(param, "TA") == 0) TA = value;
                else if (strcmp(param, "KP") == 0) KP = value;
                else if (strcmp(param, "KI") == 0) KI = value;
                else if (strcmp(param, "KD") == 0) KD = value;

                Serial.printf("[WS] %s = %.3f\n", param, value);
            }
        }
    }
}

void setup() {
    Serial.begin(115200);
    WiFi.begin("SSID", "PASSWORD");
    while (WiFi.status() != WL_CONNECTED) delay(500);

    ws.onEvent(onWsEvent);
    server.addHandler(&ws);
    server.begin();
}

...

...

**Le flux complet :**
...

Slider bougé → JSON {"param":"KP","value":1.45}
→ WebSocket → ESP32

```

```
→ deserializeJson()  
→ KP = 1.45f ✓
```

Les bibliothèques nécessaires dans `platformio.ini` :

ini

```
lib_deps =  
  ESP Async WebServer  
  ArduinoJson
```

Tu veux que j'intègre aussi la connexion WebSocket dans l'interface HTML, avec reconnexion automatique et indicateur WIFI lié à l'état réel de la socket ?

oui, et je voudrais sauvegarder,(en cliquant sur le bouton SAVE ON SD sur une carte SD associée à l'ESP32) un fichier PID.json qui contient les 4 valeurs des curseurs ainsi que les valeurs Min et Max. De même au chargement de la page on charge le fichier PID.json de la carte SD (présent dans le répertoire /balrob) et on affecte les valeurs aux champs correspondants et on place les curseurs en conséquence. { "TAmin": 80, "TAmx": 100, "TAval": 90, "KPmin": 10, "KPmax": 50, "KPval": 25, "KImin": 2, "KImax": 5, "KIval": 3, "KDmin": 1, "KDmax": 3, "KDval": 2 }

Les valeurs TAval, KPval,KIval,KDval sont envoyées au programme C++ dans les variables TA,Kp,KI,KD

06:37

Voici les deux parties complètes — le widget mis à jour et le code C++ complet.

Et voici le code C++ complet pour l'ESP32 :

cpp

```

#include <Arduino.h>
#include <WiFi.h>
#include <ESPAsyncWebServer.h>
#include <ArduinoJson.h>
#include <SD.h>
#include <SPI.h>

// — Credentials —————
const char* SSID      = "TON_SSID";
const char* PASSWORD = "TON_PASSWORD";

// — Brochage SD (adapter selon ton câblage) —————
#define SD_CS  5

// — Variables PID globales —————
float TA = 90.0f;
float KP = 25.0f;
float KI =  3.0f;
float KD =  2.0f;

// — Chemin fichier —————
const char* PID_PATH = "/balrob/PID.json";

AsyncWebServer server(80);
AsyncWebsocket ws("/ws");

// —————
// Lecture PID.json depuis la SD
// —————
bool loadPidFromSD() {
    if (!SD.exists(PID_PATH)) {
        Serial.println("[SD] PID.json introuvable");
        return false;
    }
    File f = SD.open(PID_PATH, FILE_READ);
    if (!f) return false;

    StaticJsonDocument<512> doc;
    if (deserializeJson(doc, f) != DeserializationError::Ok) {
        Serial.println("[SD] Erreur parsing JSON");
        f.close(); return false;
    }
    f.close();

    TA = doc["TAval"] | TA;
    KP = doc["KPval"] | KP;
    KI = doc["KIval"] | KI;
    KD = doc["KDval"] | KD;

    Serial.printf("[SD] Chargé: TA=%.2f KP=%.2f KI=%.2f KD=%.2f\n", TA, KP, KI, KD);
    return true;
}

// —————
// Écriture PID.json sur la SD
// —————

```

```

bool savePidToSD(JsonObject& data) {
    // Créer le répertoire si nécessaire
    if (!SD.exists("/balrob")) SD.mkdir("/balrob");

    File f = SD.open(PID_PATH, FILE_WRITE);
    if (!f) {
        Serial.println("[SD] Impossible d'ouvrir PID.json en écriture");
        return false;
    }

    StaticJsonDocument<512> doc;
    // Recopier les champs reçus du navigateur
    for (JsonPair kv : data) doc[kv.key()] = kv.value();

    serializeJsonPretty(doc, f);
    f.close();
    Serial.println("[SD] PID.json sauvegardé");
    return true;
}

// -----
// Gestionnaire WebSocket
// -----
void onWsEvent(AsyncWebSocket* server, AsyncWebSocketClient* client,
               AwsEventType type, void* arg, uint8_t* data, size_t len) {

    if (type == WS_EVT_CONNECT) {
        Serial.printf("[WS] Client #%u connecté\n", client->id());
    }
    else if (type == WS_EVT_DISCONNECT) {
        Serial.printf("[WS] Client #%u déconnecté\n", client->id());
    }
    else if (type == WS_EVT_DATA) {
        AwsFrameInfo* info = (AwsFrameInfo*)arg;
        if (info->opcode != WS_TEXT) return;
        data[len] = '\0';

        StaticJsonDocument<512> doc;
        if (deserializeJson(doc, (char*)data) != DeserializationError::Ok) return;

        const char* cmd = doc["cmd"] | "";

        // — Mise à jour d'un paramètre —
        if (strcmp(cmd, "param") == 0) {
            const char* param = doc["param"] | "";
            float value      = doc["value"] | 0.0f;

            if (strcmp(param, "TA") == 0) TA = value;
            else if (strcmp(param, "KP") == 0) KP = value;
            else if (strcmp(param, "KI") == 0) KI = value;
            else if (strcmp(param, "KD") == 0) KD = value;

            Serial.printf("[PID] %s = %.3f\n", param, value);
        }

        // — Sauvegarde sur SD —

```

```

else if (strcmp(cmd, "save") == 0) {
    JsonObject data_obj = doc["data"].as<JsonObject>();
    bool ok = savePidToSD(data_obj);

    // Réponse au navigateur
    String resp = ok
        ? "{\"event\":\"saved\"}"
        : "{\"event\":\"error\",\"detail\":\"Echec écriture SD\"}";
    client->text(resp);
}

// — Joystick —
else if (strcmp(cmd, "joy") == 0) {
    int x      = doc["x"] | 0;
    int y      = doc["y"] | 0;
    const char* dir = doc["dir"] | "STOP";
    Serial.printf("[JOY] %s x=%d y=%d\n", dir, x, y);
    // → piloter tes moteurs ici
}
}
}

// _____
// Setup
// _____
void setup() {
    Serial.begin(115200);

    // SD
    if (!SD.begin(SD_CS)) {
        Serial.println("[SD] Initialisation échouée");
    } else {
        Serial.println("[SD] Carte SD OK");
        loadPidFromSD(); // ← charge les valeurs au démarrage
    }

    // WiFi
    WiFi.begin(SSID, PASSWORD);
    Serial.print("[WIFI] Connexion");
    while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
    Serial.printf("\n[WIFI] IP: %s\n", WiFi.localIP().toString().c_str());

    // Servir les fichiers statiques depuis /balrob sur la SD
    server.serveStatic("/balrob/", SD, "/balrob/");

    // WebSocket
    ws.onEvent(onWsEvent);
    server.addHandler(&ws);

    // Servir index.html
    server.serveStatic("/", SD, "/balrob/").setDefaultFile("index.html");

    server.begin();
    Serial.println("[HTTP] Serveur démarré");
}

```



```
void loop() {
  ws.cleanupClients();
  // → ton code PID ici, utilise TA, KP, KI, KD
}
...

**Ce qui se passe au démarrage :**
...

SD init → load /balrob/PID.json → TA/KP/KI/KD affectés
Page web chargée → fetch /balrob/PID.json → curseurs positionnés
WebSocket connecté → badge WIFI ON, bouton SAVE activé
```

Librairies `platformio.ini` :

ini

```
lib_deps =
  ESP Async WebServer
  ArduinoJson
  SD
```

Note : l'`index.html` doit lui aussi être déposé sur la SD dans `/balrob/index.html` pour que `server.serveStatic` le serve correctement.

Save this visual from earlier as an artifact: "robot_pid_ws_sd"

06:45

Voilà ! Le fichier `robot_pid_ws_sd.html` est prêt à être téléchargé. C'est la page autonome complète à déposer sur ta carte SD dans `/balrob/index.html` — elle sera servie directement par l'ESP32 via `server.serveStatic`.

Robot pid ws sd

Code · HTML

je voudrais que le bouton WIFI fasse apparaître la fenêtre choix wifi avec la possibilité de ranger(bouton save config) sur carte SD un fichier wifi.json indiquant le choix du reseau et son mot de passe : { "mode": "STA", "ssid": "mySSID", "password": "mypassword" } De meme au lancement du programme C++ on initialisera des variables avec ces valeurs lues à partir de la carte SD

10:05

Et voici le code C++ complet pour la gestion de `wifi.json` côté ESP32 :

cpp

```

#include <Arduino.h>
#include <WiFi.h>
#include <ESPAsyncWebServer.h>
#include <ArduinoJson.h>
#include <SD.h>
#include <SPI.h>

#define SD_CS 5

// — Variables WiFi globales —————
String wifiMode      = "STA";
String wifiSSID      = "";
String wifiPassword  = "";
String wifiIP        = "";

// — Chemins fichiers —————
const char* WIFI_PATH = "/balrob/wifi.json";
const char* PID_PATH  = "/balrob/PID.json";

AsyncWebServer server(80);
AsyncWebsocket ws("/ws");

// —————
// Lecture wifi.json depuis la SD
// —————
bool loadWifiFromSD() {
    if (!SD.exists(WIFI_PATH)) {
        Serial.println("[SD] wifi.json introuvable");
        return false;
    }
    File f = SD.open(WIFI_PATH, FILE_READ);
    if (!f) return false;

    StaticJsonDocument<256> doc;
    if (deserializeJson(doc, f) != DeserializationError::Ok) {
        f.close(); return false;
    }
    f.close();

    wifiMode      = doc["mode"]      | "STA";
    wifiSSID      = doc["ssid"]      | "";
    wifiPassword  = doc["password"] | "";

    Serial.printf("[SD] WiFi chargé: mode=%s ssid=%s\n",
                  wifiMode.c_str(), wifiSSID.c_str());
    return true;
}

// —————
// Sauvegarde wifi.json sur la SD
// —————
bool saveWifiToSD(const char* mode, const char* ssid, const char* password) {
    if (!SD.exists("/balrob")) SD.mkdir("/balrob");
    File f = SD.open(WIFI_PATH, FILE_WRITE);
    if (!f) return false;

```

```

StaticJsonDocument<256> doc;
doc["mode"]      = mode;
doc["ssid"]      = ssid;
doc["password"]  = password;

serializeJsonPretty(doc, f);
f.close();
Serial.println("[SD] wifi.json sauvegardé");
return true;
}

// -----
// Connexion WiFi selon le mode
// -----
void startWifi() {
    WiFi.disconnect(true);
    delay(100);

    if (wifiMode == "AP") {
        WiFi.softAP(wifiSSID.c_str(), wifiPassword.c_str());
        wifiIP = WiFi.softAPIP().toString();
        Serial.printf("[WIFI] AP démarré – IP: %s\n", wifiIP.c_str());
    } else {
        WiFi.mode(WIFI_STA);
        WiFi.begin(wifiSSID.c_str(), wifiPassword.c_str());
        Serial.print("[WIFI] Connexion STA");
        uint8_t tries = 0;
        while (WiFi.status() != WL_CONNECTED && tries++ < 20) {
            delay(500); Serial.print(".");
        }
        if (WiFi.status() == WL_CONNECTED) {
            wifiIP = WiFi.localIP().toString();
            Serial.printf("\n[WIFI] Connecté → IP: %s\n", wifiIP.c_str());
        } else {
            Serial.println("\n[WIFI] Échec connexion");
        }
    }
}

// -----
// WebSocket – gestionnaire d'événements
// -----
void onWsEvent(AsyncWebSocket* srv, AsyncWebSocketClient* client,
               AwsEventType type, void* arg, uint8_t* data, size_t len) {

    if (type != WS_EVT_DATA) return;
    AwsFrameInfo* info = (AwsFrameInfo*)arg;
    if (info->opcode != WS_TEXT) return;
    data[len] = '\0';

    StaticJsonDocument<1024> doc;
    if (deserializeJson(doc, (char*)data) != DeserializationError::Ok) return;

    const char* cmd = doc["cmd"] | "";

    // — Paramètre PID —

```

```

if (strcmp(cmd, "param") == 0) {
    const char* param = doc["param"] | "";
    float value = doc["value"] | 0.0f;
    if (strcmp(param, "TA") == 0) TA = value;
    else if (strcmp(param, "KP") == 0) KP = value;
    else if (strcmp(param, "KI") == 0) KI = value;
    else if (strcmp(param, "KD") == 0) KD = value;
    Serial.printf("[PID] %s = %.3f\n", param, value);
}

// — Sauvegarde PID.json —
else if (strcmp(cmd, "save") == 0) {
    JsonObject d = doc["data"].as<JsonObject>();
    bool ok = savePidToSD(d);
    client->text(ok
        ? "{\"event\":\"saved\", \"file\":\"PID.json\"}"
        : "{\"event\":\"error\", \"detail\":\"Echec SD\"}");
}

// — Sauvegarde wifi.json —
else if (strcmp(cmd, "save_wifi") == 0) {
    JsonObject d = doc["data"].as<JsonObject>();
    const char* mode = d["mode"] | "STA";
    const char* ssid = d["ssid"] | "";
    const char* pass = d["password"] | "";
    bool ok = saveWifiToSD(mode, ssid, pass);
    client->text(ok
        ? "{\"event\":\"saved\", \"file\":\"wifi.json\"}"
        : "{\"event\":\"error\", \"detail\":\"Echec SD wifi\"}");
}

// — Scan Wi-Fi —
else if (strcmp(cmd, "wifi_scan") == 0) {
    int n = WiFi.scanNetworks();
    StaticJsonDocument<1024> resp;
    resp["event"] = "wifi_scan_result";
    JsonArray arr = resp.createNestedArray("networks");
    for (int i = 0; i < n; i++) {
        JsonObject net = arr.createNestedObject();
        net["ssid"] = WiFi.SSID(i);
        net["rssi"] = WiFi.RSSI(i);
        net["secure"] = (WiFi.encryptionType(i) != WIFI_AUTH_OPEN);
    }
    String out; serializeJson(resp, out);
    client->text(out);
}

// — Connexion Wi-Fi à la volée —
else if (strcmp(cmd, "wifi_connect") == 0) {
    wifiMode = doc["mode"] | "STA";
    wifiSSID = doc["ssid"] | "";
    wifiPassword = doc["password"] | "";
    startWifi();

    String resp;
    if (WiFi.status() == WL_CONNECTED || wifiMode == "AP") {

```

```

        StaticJsonDocument<128> r;
        r["event"] = "wifi_connected";
        r["ssid"] = wifiSSID;
        r["ip"] = wifiIP;
        serializeJson(r, resp);
    } else {
        resp = "{\"event\":\"wifi_failed\"}";
    }
    client->text(resp);
}

// — Joystick —
else if (strcmp(cmd, "joy") == 0) {
    int x = doc["x"] | 0, y = doc["y"] | 0;
    Serial.printf("[JOY] %s x=%d y=%d\n", doc["dir"]|"", x, y);
    // → pilotage moteurs
}
}

// _____
// Setup
// _____
void setup() {
    Serial.begin(115200);

    if (!SD.begin(SD_CS)) {
        Serial.println("[SD] Carte SD KO");
    } else {
        Serial.println("[SD] Carte SD OK");
        loadWifiFromSD(); // ← lit mode, ssid, password
        loadPidFromSD(); // ← lit TA, KP, KI, KD
    }

    startWifi(); // ← démarre en STA ou AP selon wifi.json

    server.serveStatic("/balrob/", SD, "/balrob/");
    server.serveStatic("/", SD, "/balrob/").setDefaultFile("index.html");
    ws.onEvent(onWsEvent);
    server.addHandler(&ws);
    server.begin();
}

void loop() {
    ws.cleanupClients();
    // → boucle PID avec TA, KP, KI, KD
}
...

**Résumé du flux WiFi : **
...

Démarrage ESP32
→ lit wifi.json → wifiMode / wifiSSID / wifiPassword
→ startWifi() → STA (se connecte) ou AP (crée un hotspot)

Bouton WIFI dans la page
→ charge wifi.json par HTTP pour pré-remplir les champs

```

- [Scanner] → cmd wifi_scan → ESP32 scanne → liste des réseaux
- [Save config] → cmd save_wifi → écrit wifi.json sur SD
- [Connecter] → cmd wifi_connect → ESP32 se reconnecte à chaud

idem avec le bouton FILTER et un fichier filter.json : { "filter": "MOYENNE", "interval": 500 }

Chargé au démarrage et enregistré avec le bouton SAVE FILTER

10:13

Et voici les ajouts C++ pour `filter.json` :

cpp

```

// — Variables filtre globales —————
String filterType      = "MOYENNE";
int    filterInterval = 500;

const char* FILTER_PATH = "/balrob/filter.json";

// —————
// Lecture filter.json depuis la SD
// —————
bool loadFilterFromSD() {
    if (!SD.exists(FILTER_PATH)) {
        Serial.println("[SD] filter.json introuvable");
        return false;
    }
    File f = SD.open(FILTER_PATH, FILE_READ);
    if (!f) return false;

    StaticJsonDocument<128> doc;
    if (deserializeJson(doc, f) != DeserializationError::Ok) {
        f.close(); return false;
    }
    f.close();

    filterType      = doc["filter"]      | "MOYENNE";
    filterInterval = doc["interval"] | 500;

    Serial.printf("[SD] Filtre: %s · %dms\n",
                  filterType.c_str(), filterInterval);
    return true;
}

// —————
// Sauvegarde filter.json sur la SD
// —————
bool saveFilterToSD(const char* filter, int interval) {
    if (!SD.exists("/balrob")) SD.mkdir("/balrob");
    File f = SD.open(FILTER_PATH, FILE_WRITE);
    if (!f) return false;

    StaticJsonDocument<128> doc;
    doc["filter"]      = filter;
    doc["interval"] = interval;

    serializeJsonPretty(doc, f);
    f.close();
    Serial.println("[SD] filter.json sauvegardé");
    return true;
}

// —————
// Application du filtre (à adapter à ton algo)
// —————
void applyFilter() {
    if (filterType == "MOYENNE") { /* moyenne glissante sur N échantillons */ }
    else if (filterType == "MEDIAN") { /* filtre médian */ }
    else if (filterType == "PASSE_BAS") { /* filtre passe-bas (RC numérique) */ }
}

```



```
// filterInterval → période d'échantillonnage en ms
}
```

Dans le gestionnaire WebSocket, ajouter les deux cas :

cpp

```
// — Sauvegarde filter.json —
else if (strcmp(cmd, "save_filter") == 0) {
    const char* filter = doc["data"]["filter"] | "MOYENNE";
    int interval      = doc["data"]["interval"] | 500;
    bool ok = saveFilterToSD(filter, interval);
    client->text(ok
        ? "{\"event\":\"saved\", \"file\":\"filter.json\"}"
        : "{\"event\":\"error\", \"detail\":\"Echec SD filter\"}");
}

// — Application immédiate du filtre —
else if (strcmp(cmd, "filter_apply") == 0) {
    filterType      = doc["filter"] | "MOYENNE";
    filterInterval = doc["interval"] | 500;
    Serial.printf("[FILTER] Appliqué: %s · %dms\n",
        filterType.c_str(), filterInterval);
    applyFilter();
}
```

Dans `setup()` :

cpp

```
loadFilterFromSD(); // ← après SD.begin()
applyFilter();      // ← applique immédiatement
```

Ce qui est visible dans l'interface :

- Le badge `MOYENNE · 500ms` à côté du bouton FILTER se met à jour en temps réel
- La ligne `Enregistrement en cours...` apparaît pendant la sauvegarde, puis `✓ Sauvegardé` à la confirmation de l'ESP32
- SAVE FILTER envoie aussi `filter_apply` pour appliquer sans redémarrer

expliquer le code qui reroute la ligne série vers le terminal de la page web

11:06

Il n'y a pas encore de rerouting de la ligne série dans le code actuel — le terminal affiche uniquement les messages générés par le JavaScript lui-même (actions sur les sliders, joystick, etc.). Voici comment implémenter le vrai reroutage `Serial → WebSocket → terminal web`.

Le principe

ESP32

Navigateur

`Serial.printf(...)` —WS→ `ws.onmessage` → `addLog(...)`

Le `Serial` normal écrit sur l'UART. Il faut intercepter ces writes et les dupliquer vers le WebSocket.

Côté C++ — classe SerialOverWS

cpp

```

// -----
// Classe qui hérite de Print (base de Serial)
// et redirige chaque write() vers le WebSocket
// -----
class SerialOverWS : public Print {
public:
    AsyncWebSocket* _ws;
    String          _buf;    // buffer ligne courante

    SerialOverWS(AsyncWebSocket* ws) : _ws(ws) {}

    // Appelé pour CHAQUE caractère ou bloc écrit
    size_t write(uint8_t c) override {
        // On accumule dans le buffer jusqu'au '\n'
        if (c == '\n') {
            flush();
        } else {
            _buf += (char)c;
        }
        return 1;
    }

    size_t write(const uint8_t* buf, size_t size) override {
        for (size_t i = 0; i < size; i++) write(buf[i]);
        return size;
    }

    // Envoie la ligne accumulée à tous les clients WS
    void flush() {
        if (_buf.length() == 0) return;

        // Format JSON attendu par le terminal JS
        StaticJsonDocument<256> doc;
        doc["event"] = "serial";
        doc["msg"]   = _buf;

        String out;
        serializeJson(doc, out);
        _ws->textAll(out);    // broadcast à tous les clients connectés

        _buf = "";           // vide le buffer
    }
};

// Instance globale
SerialOverWS* serialWS = nullptr;

// Macro pratique : écrit sur Serial ET sur le WS
#define LOG(fmt, ...) {                                     \
    Serial.printf(fmt, ##__VA_ARGS__);                     \
    if (serialWS) serialWS->printf(fmt, ##__VA_ARGS__); \
}

```

Dans `setup()`, après avoir créé le WebSocket :

cpp

```

serialWS = new SerialOverWS(&ws);

// Maintenant on peut utiliser LOG() partout
LOG("[WiFi] Connecté → IP: %s\n", WiFi.localIP().toString().c_str());
LOG("[PID] TA=%.1f KP=%.3f KI=%.3f KD=%.3f\n", TA, KP, KI, KD);
LOG("[IMU] angle=%.2f° err=%.2f°\n", angle, error);

```

Côté JavaScript — réception dans `ws.onmessage`

Dans le gestionnaire déjà existant, ajouter le cas `serial` :

javascript

```

ws.onmessage = e => {
  try {
    const msg = JSON.parse(e.data);

    if (msg.event === 'serial') {
      // Message venant de Serial/LOG() sur l'ESP32
      addLog('[ESP32] ' + msg.msg, '');
    }
    else if (msg.event === 'saved') { ... }
    else if (msg.event === 'error') { ... }
    // etc.
  } catch(_) {
    addLog('[WS] ' + e.data, '');
  }
};
...

---

## Flux complet, pas à pas
...

1. Code C++ appelle :
   LOG("[IMU] angle=89.7° err=0.3°\n")

2. SerialOverWS::write() reçoit chaque caractère
   → accumule dans _buf : "[IMU] angle=89.7° err=0.3°"
   → '\n' détecté → flush()

3. flush() sérialise en JSON :
   {"event":"serial","msg":"[IMU] angle=89.7° err=0.3°"}
   → ws.textAll() → envoyé à tous les navigateurs connectés

4. js ws.onmessage reçoit le JSON
   → msg.event === 'serial'
   → addLog('[ESP32] [IMU] angle=89.7° err=0.3°', '')

5. addLog() crée une <div class="term-line"> horodatée
   → insérée dans #terminal → scrollTop mis à jour

```

Pourquoi passer par une classe `Print` ?

`Serial` sur Arduino/ESP32 est lui-même une classe qui hérite de `Print`. La classe `Print` définit une interface avec une seule méthode abstraite : `write(uint8_t)`. Tout le reste (`printf`, `println`, `print`) est construit par-dessus. En héritant de `Print` et en redéfinissant uniquement `write()`, on intercepte **toutes** les formes d'écriture sans modifier le reste du code.

est il possible de rerouter également les messages vers la console de javascript

11:09

Oui, c'est très simple — il suffit d'ajouter un `console.log()` dans le même endroit où on reçoit le message `serial`, côté JavaScript.

Modification dans `ws.onmessage`

javascript

```
ws.onmessage = e => {
  try {
    const msg = JSON.parse(e.data);

    if (msg.event === 'serial') {
      // — Terminal web —
      addLog('[ESP32] ' + msg.msg, '');

      // — Console JavaScript —
      console.log('%c[ESP32] ' + msg.msg, 'color: #378ADD; font-family: monospace;');
    }
    else if (msg.event === 'saved') {
      addLog('[SD] ' + msg.file + ' sauvegardé ✓', 'ok');
      console.info('[SD] ' + msg.file + ' sauvegardé');
    }
    else if (msg.event === 'error') {
      addLog('[SD] Erreur: ' + msg.detail, 'err');
      console.error('[SD] ' + msg.detail);
    }
    else if (msg.event === 'wifi_connected') {
      addLog('[WIFI] Connecté → ' + msg.ssid + ' (' + msg.ip + ')', 'ok');
      console.info('[WIFI] Connecté →', msg.ssid, msg.ip);
    }
  } catch(_) {
    addLog('[WS] ' + e.data, '');
    console.log('[WS raw]', e.data);
  }
};
```

Les niveaux de console disponibles

Tu peux choisir le niveau selon la sévérité du message :

javascript

```
console.log('...')    // message normal        → fond blanc
console.info('...')   // information            → icône ⓘ
console.warn('...')   // avertissement          → fond jaune
console.error('...')  // erreur                  → fond rouge
console.debug('...')  // debug (masqué par défaut, filtre "Verbose")
```

Mise en forme avec `%c` et couleurs

javascript

```
// Préfixe coloré selon la catégorie du message
function consoleLog(msg) {
  if (msg.startsWith('[IMU]')) console.log('%c' + msg, 'color:#1D9E75; font-family:monospace;');
  else if (msg.startsWith('[PID]')) console.log('%c' + msg, 'color:#378ADD; font-family:monospace;');
  else if (msg.startsWith('[WIFI]')) console.log('%c' + msg, 'color:#BA7517; font-family:monospace;');
  else if (msg.startsWith('[ERR]')) console.error(msg);
  else if (msg.startsWith('[WARN]')) console.warn(msg);
  else console.log('%c' + msg, 'color:#5F5E5A; font-family:monospace;');
}
```

Groupe pour ne pas noyer la console

Si l'ESP32 envoie beaucoup de messages (boucle PID rapide), tu peux les regrouper :

javascript

```
// Regroupe les messages par catégorie dans un groupe rétractable
function consoleLog(msg) {
  const category = msg.match(/^\\[(\\w+)\\]/)?.[1] || 'ESP32';
  console.groupCollapsed('%c[' + category + ']', 'color:#378ADD; font-weight:500;');
  console.log(msg);
  console.groupEnd();
}
...
```

```
## Résultat dans les DevTools (F12)
...
```

```
● [ESP32] [PID] TA=90.0 KP=1.200 KI=0.050 KD=0.300
● [ESP32] [IMU] angle=89.7° err=0.3°
⚠ [ESP32] [WARN] Batterie faible: 3.6V
● [ESP32] [ERR] SD write failed
```

Le double affichage — terminal web **et** console JS — est utile pendant le développement : la console JS conserve l'historique complet même si le terminal web est limité à 100 lignes, et elle est accessible même si la page web est rechargée.

et maintenant rerouter les message de la console vers le terminal de la page web

11:10

Il suffit d'**override les méthodes natives de** `console` au tout début du script, avant tout autre code.

Le code — à placer en tête de script

javascript

```

// -----
// Reroutage console → terminal web
// Doit être déclaré AVANT tout autre code
// -----
(function interceptConsole() {
  const methods = {
    log:    '',
    info:   'ok',
    warn:   'warn',
    error:  'err',
    debug:  ''
  };

  Object.entries(methods).forEach(([method, cls]) => {
    const original = console[method].bind(console); // garde l'original

    console[method] = function (...args) {
      // 1. Comportement normal – DevTools reçoit quand même le message
      original(...args);

      // 2. Formatage en texte lisible
      const text = args.map(a => {
        if (typeof a === 'object') {
          try { return JSON.stringify(a); }
          catch (_) { return String(a); }
        }
        return String(a);
      }).join(' ');

      // 3. Préfixe selon le niveau
      const prefix = {
        log:    '[JS]',
        info:   '[INFO]',
        warn:   '[WARN]',
        error:  '[ERR]',
        debug:  '[DBG]'
      }[method];

      // 4. Envoi au terminal (addLog peut ne pas encore exister
      //    au moment du premier appel → on protège avec un try)
      try { addLog(prefix + ' ' + text, cls); }
      catch (_) { /* terminal pas encore prêt */ }
    };
  });
})();
```

Flux complet une fois en place


```

```
console.log('test')
 → original console.log() → DevTools (inchangé)
 → addLog('[JS] test', '') → terminal web

console.warn('batterie faible')

```


```


→ original console.warn() → DevTools (fond jaune)
→ addLog('[WARN] batterie faible', 'warn') → terminal web (texte orange)

```
console.error('SD write failed')
```

→ original console.error() → DevTools (fond rouge)
→ addLog('[ERR] SD write failed', 'err') → terminal web (texte rouge)

Capturer aussi les erreurs non gérées

javascript

```
// Exceptions JavaScript non catchées
window.addEventListener('error', e => {
  try { addLog('[EXCEPTION] ' + e.message + ' (' + e.filename + ':' + e.lineno + ')',
    'err'); }
  catch (_) {}
});
```

```
// Promesses rejetées sans .catch()
window.addEventListener('unhandledrejection', e => {
  try { addLog('[PROMISE] ' + String(e.reason), 'err'); }
  catch (_) {}
});
...
---
```

Résultat dans le terminal web

```
...
10:12:01 [JS] [WS] Connexion → ws://192.168.1.42/ws
10:12:01 [INFO] [WS] Connecté → 192.168.1.42
10:12:02 [JS] [SD] PID.json chargé
10:12:05 [JS] [PID] KP = 1.45
10:12:10 [WARN] [WS] Déconnecté – reconnexion 3s
10:12:13 [ERR] [EXCEPTION] Cannot read property 'value' (index.html:142)
...
---
```

Récapitulatif – les 3 sources dans le terminal

Source	Préfixe	Déclencheur
ESP32 via WebSocket	`[ESP32]`	`LOG(...)` côté C++
Console JavaScript	`[JS]` ` [WARN]` ` [ERR]`	`console.log/warn/error()`
Erreurs non gérées	`[EXCEPTION]` ` [PROMISE]`	crash JS silencieux

L'ordre dans le script doit être :

- ```
...
1. interceptConsole() ← en premier, avant tout
2. window.addEventListener('error', ...)
3. window.addEventListener('unhandledrejection', ...)
4. ... reste du code (addLog, WebSocket, PID, joystick...)
```

